Project Blueprint

An Immersive, 3D Interactive Web Experience Proposal

Prepared for: Private Digital Commission

System Component Documents: PRD, TRD, App Flow, UI/UX Brief, Schema, Implementation Plan

Design Theme: Elegant Pinterest-Inspired Floral Realism

Target Environments: Modern Web Browsers (WebGL-enabled)

Document 1: Product Requirement Document (PRD)

1.1 Executive Summary & Core Concept

The objective of this project is to develop a premium, bespoke digital proposal web application. The core conceptual hook is built around the sentiment: *"Flowers that never die."* Leveraging custom 3D web-native rendering, interactive canvas interfaces, and editorial web design typography, the application transitions a deeply personal history into a curated narrative experience.

1.2 Target Audience & UX Expectations

The sole primary user is an individual with high aesthetic standards, heavily influenced by editorial, minimalist, and romantic design layouts commonly found on platforms like Pinterest. The application must avoid standard tech-centric layout models (such as hero blocks, stark modern cards, or navigation bars) and instead present itself as a fluid, responsive, artistic gallery.

1.3 Functional Requirements & Feature Matrix

Phase / State	Functional Requirement	Priority
1. Preload & Init	Asset preloader showcasing an elegant custom loading text. No sudden frame shifts. WebGL context initialization.	Critical
2. Interactive Landing	Procedurally floating 3D tulips experiencing infinite slow drift. Mouse/Touch cursor interaction forces real-time localized displacement.	Critical
3. Call-to-Action	Centralized HTML "Reveal" button with soft fade styling. Triggers complex transition tween loop on click.	Critical
4. Framing Transition	Tulips gracefully move from physics/drift coordinates to static layout grid locations, framing the edges of the active display viewport.	High
5. Storytelling Timeline	Vertical text and visual container fade-ins controlled exclusively via user scroll activity. Dynamic scroll tracking.	Critical
6. Finale & Response	The final layout node anchors a single editorial inquiry paired with premium low-friction selection items.	Critical

1.4 Key Performance Indicators (KPIs)

- **Render Stability:** Constant 60 Frames Per Second (FPS) on standard target hardware.

- **Load Time:** Visual presentation begins within 3.5 seconds over standard modern mobile broadband connections.
- **Responsiveness:** Absolute functional parity across standard desktop monitor setups and modern mobile web browser views.

Document 2: Technical Requirement Document (TRD)

2.1 Core System Architecture

The implementation will leverage a modern, client-side frontend stack to guarantee extreme performance stability without unnecessary structural overhead. The software ecosystem is explicitly detailed below:

> **Selected Framework & Compilation Engine:** Vite + Vanilla JavaScript Suite.
> **3D Graphics Engine:** Three.js (via raw WebGL context bindings).
> **Physics Engine:** Matter.js (2D bounding calculations linked to 3D positioning matrices).
> **Animation & Scroll Driver:** GSAP (GreenSock Animation Platform) containing Core, ScrollTrigger, and CustomEase libraries.

2.2 Physics Mechanics: Cursor Repulsion Logic

To establish tactile interaction, the cursor coordinate matrix translates physical viewport offsets directly into spatial fields. Let the cursor coordinate position be represented as $P_c = (x_c, y_c)$ and any given 3D element instance vector position be represented as $P_i = (x_i, y_i)$. The relative directional displacement vector D is determined via:

$$D = P_i - P_c$$

The raw scalar distance $r = ||D||$ establishes the interaction threshold. If $r < R_{threshold}$, a repulsion acceleration vector force F is applied inversely proportional to proximity:

$$F = (k \times (R_{threshold} - r) / r) \times D$$

Where k represents the customizable physical spring coefficient. This ensures that as the user's cursor approaches an element, it accelerates cleanly away into open visual space.

2.3 Resource Optimization Strategy

- **Asset Compression:** All 3D models must utilize the efficient `.glb` container scheme and run through Drury-compressed optimization steps to limit overall weight to under 2.5MB per model entity.
- **Texture Mapping:** Use specialized 1024x1024 desaturated texture profiles paired with ambient occlusion maps to maintain realistic visual depth without triggering runtime GPU lag.

Document 3: Application Flow & Interaction Model

3.1 High-Level Architecture Flowchart

The sequential execution structure of the web interface follows a rigid structural sequence, detailed through the logical state flow outlined below:

```
[ State 01: Preloader Initialization ]
               |
               ▼ (Assets Cached & WebGL Handshake Confirmed)
[ State 02: Interactive Landing Canvas ] ◄──► [ Mouse Move / Touch Event Triggers ]
               |
               ▼ (User Interacts with "Reveal" Node)
[ State 03: Framing Animation State ] ──► (GSAP Coordinates Recalculated)
               |
               ▼ (Viewport Layout Transition Complete)
[ State 04: Narrative Scroll Sequence ] ◄──► [ ScrollTrigger Input Matrix ]
               |
               ▼ (Terminal Boundary Encountered)
[ State 05: Finale Inversion Mode ]
```

3.2 State Traversal Explanations

1. **State 01 (Preloader):** Blocks background interaction. Initializes modern canvas textures into memory. Fades out instantly upon loading completion.
2. **State 02 (Interactive Landing Canvas):** Binds mouse coordinate movement vectors to the active physics layer. The system queries cursor position at a locked 60Hz evaluation rhythm.
3. **State 03 (Framing Animation State):** Kills the active mouse-collision runtime loop cleanly. Uses GSAP parameters to transform individual 3D objects to structural viewport boundary layout positions.
4. **State 04 (Narrative Scroll Sequence):** Mounts standard DOM scrolling mechanics. CSS elements glide over the background. Camera angles scale smoothly based on scroll metrics.
5. **State 05 (Finale Inversion Mode):** The user scroll depth meets absolute limits, bringing visibility to interaction controls to complete the functional narrative cycle.

Document 4: UI/UX Design Brief

4.1 Visual System & Artistic Direction

The design archetype leans heavily on premium, high-end editorial layouts. The visual space must breathe naturally, prioritizing wide margins, expansive layouts, and deliberate content placement over raw information density. Visual interactions must feel smooth and fluid rather than abrupt or snap-based.

4.2 Color Palette Spectrum

Color Application	Hex Coordinate Value	Aesthetic Intent
Primary Canvas	`#fcfaf7`	Warm, desaturated organic alabaster. Replaces cold digital whites.
Primary Typography	`#3b312c`	Deep earthy espresso. Provides striking, readable editorial contrast.
Secondary Text	`#8c7b72`	Soft taupe mist. Ideal for sub-headers and technical meta labels.
Floral Accent Low	`#e6cfc3`	Warm, dusty pastel blush pink used on 3D materials.
Floral Accent High	`#c9b0a0`	Muted organic rose-gold shadows to emphasize dimensional depth.

4.3 Typography Architecture

- **Display & Headline Elements:** *Georgia* (or premium editorial alternates like *Playfair Display*). Weight: Regular / Light Italic. Tracking: -0.5px. Used to communicate elegance and warmth.
- **Structural, Body, & UI Text:** *Helvetica Neue* (or clean modern variants like *Inter*). Weight: Light (300) / Regular (400). Tracking: +1px. Letter-casing: Upper-case for system UI interactive indicators.

4.4 Layout Framing & Micro-Interactions

All button structures must eschew standard borders or flat color blocks. Use fine 1px lines combined with high text tracking. Hover elements trigger ultra-soft, slow transitions (e.g., `transition: all 0.6s cubic-bezier(0.16, 1, 0.3, 1);`). Photographic assets use clean rectangular cuts with subtle parallax behavior applied during scroll events to preserve an elegant, editorial feel.

Document 5: Frontend State & Architecture Schema

5.1 Local Application State Architecture

Because this experience is completely private, self-contained, and running entirely within a local client environment, traditional backend processing layers are discarded. Data structures are managed dynamically in-memory via high-speed, localized JavaScript structures.

5.2 Data Object Interface Matrix

```
const ApplicationStateStore = {
    // Structural Metadata
    session: {
        loadingProgress: 0.00,       // Float value from 0.00 to 1.00
        currentInterfaceState: "INIT", // "INIT" | "LANDING" | "TRANSITION" | "STORY" | "FINALE"
        deviceTypology: "DESKTOP"     // "DESKTOP" | "MOBILE_PORTRAIT" | "MOBILE_LANDSCAPE"
    },

    // Cursor Vector Positions
    pointerCoordinates: {
        currentX: 0.00,
        currentY: 0.00,
        smoothedX: 0.00,              // Linear interpolation (lerp) tracking position
        smoothedY: 0.00,
        velocityMagnitude: 0.00       // Used to scale physical displacement fields
    },

    // 3D Object Model Transform Cache
    flowerEntities: [
        {
            instanceIdentifier: "tulip_node_01",
            spatialVector3D: { x: 0.0, y: 0.0, z: 0.0 },
            rotationMatrix: { x: 0.0, y: 0.0, z: 0.0 },
            velocityVector2D: { dx: 0.0, dy: 0.0 },
            anchorBoundaryTarget: { x: -5.0, y: 8.0, z: -2.0 } // Layout target coordinates
        }
    ],

    // Storytelling Timeline Tracker
    narrativeProgress: {
        scrollPercentNormalized: 0.0000, // Normalized metric from 0.0000 to 1.0000
        activeSectionIndex: 0,
        cameraInterpolationZ: 15.0       // Z-depth calculation driven by scroll values
    }
};
```

5.3 Event Handlers & Core Methods

- `initializeEngine()`: Prepares the WebGL scene wrapper and sets up responsive window sizing listeners.
- `evaluatePhysicsCalcululations(pointerX, pointerY)`: Runs frame-by-frame calculations mapping cursor-to-flower distance vectors to apply acceleration adjustments.
- `executeInterfaceTransition()`: Safely shuts down the background physics loop and runs GSAP animations to reposition flowers along the layout borders.
- `synchronizeScrollProgress(scrollY)`: Links native DOM scroll metrics to camera target positions to drive the visual timeline.

Document 6: Implementation Plan (AI Prompt Pipeline)

6.1 Phased Execution Methodology

This breakdown outlines a structured, step-by-step approach designed to build the application iteratively. You can feed each phase instruction set directly into an advanced AI development tool (like Claude 3.5 Sonnet) to write, test, and refine the code cleanly without breaking features along the way.

6.2 Phase-by-Phase Prompt Directives

Phase 1: Environment & Engine Setup Boilerplate

```
AI Prompt Specification Directive:
"Write a modern, single-file frontend configuration using standard index.html and a modular application script. Pull in Three.js, GSAP 3.x, and Matter.js via secure CDN references. Initialize a full-screen, responsive 3D WebGL render layer utilizing a transparent configuration layer set precisely at `#fcfaf7`. Add standard window resizing calculations to update the aspect ratio automatically, and establish an empty 60fps animation render loop."
```

Phase 2: 3D Object Injection & Physics Engine Implementation

```
AI Prompt Specification Directive:
"Building on top of our previous framework, initialize a Matter.js 2D physics world. Programmatically generate 30 floating object nodes mapped to the screen's canvas dimensions. Bind global pointer event listeners to capture exact cursor coordinate updates. Apply smooth linear interpolation (lerp) to smooth out cursor movement, and implement real-time repulsion vectors so that objects slide cleanly away from the cursor path when it crosses their boundary radius."
```

Phase 3: Centralized CTA UI & Transition Logic

```
AI Prompt Specification Directive:
"Construct a clean HTML overlay container centered on the screen. Add an elegant button with text reading 'REVEAL'. Style the button with letter-spacing details matching an elegant, minimalist Pinterest design. Implement a click-event listener that safely stops the active Matter.js physics tracking loops and uses high-performance GSAP tweens to smoothly animate each 3D node from its current physics position out to the visual margins of the screen, creating a clean frame border."
```

Phase 4: Scroll-Driven Storytelling Timeline

```
AI Prompt Specification Directive:
"Now that the 3D nodes are locked into framing positions along the edges of the viewable screen, mount a scrollable vertical DOM layout structure. Integrate GSAP ScrollTrigger to track page scroll metrics. As the user moves down the page, trigger smooth fade-in animations for alternating text sections and images. Maintain an elegant, airy layout format with generous vertical spacing and clean serif typography throughout the scroll experience."
```

Phase 5: Finale Integration & Mobile Review Optimization

```
AI Prompt Specification Directive:
"At the ultimate bottom boundary of the scroll layout, introduce the final text element question, styled using premium serif typography. Below this question, display two high-contrast minimalist interface response toggles. Ensure all interaction code handles touch inputs gracefully, and double-check responsive scaling calculations so the layout works flawlessly across standard modern mobile displays."
```